TabCL: Continual Malware Classification with Tabular-Aware Generation

Jimin Park[1], Haeseung Jeon[1], AHyun Ji[1], Aritran Piplai[2], Mohammad Saidur Rahman[2], and Se Eun Oh[1(✉)]

[1] Division of Artificial Intelligence and Software, Ewha Womans University, Seoul, South Korea
{242aig12,haeseungjeon,nolli77,seoh}@ewha.ac.kr
[2] University of Texas at El Paso, El Paso, USA
{apiplai,msrahman3}@utep.edu

Abstract. Despite advances in machine learning (ML)-based malware detection, the fundamental ML weakness of catastrophic forgetting (CF) has not yet been actively addressed, even though it poses a critical challenge due to the continual emergence of new malware families. While recent work such as MalCL applies generative replay (GR) using Generative Adversarial Networks (GANs), their approach does not account for the unique structural characteristics of tabular malware features or the computational burden of handling extremely high-dimensional feature spaces. To address these challenges, we introduce TabCL, an improved continual learning (CL) framework that adopts a conditional tabular GAN (CTGAN) for malware classification. Furthermore, to reduce the complexity of high-dimensional malware samples, we propose SiMBER, a SimHash-based feature compressor that reduces dimensionality by approximately 60% while preserving or even enhancing classification performance. By integrating a CTGAN with SiMBER, TabCL enables structure-aware modeling of malware samples across both continuous and categorical features while improving training efficiency. TabCL achieved over a 10% accuracy improvement and consistently outperforms all baselines across EMBER 2018, EMBER 2024, and MalDICT (The source code is available on https://github.com/AISec-EWHA/TabCL.).

Keywords: Continual Learning · Malware Analysis · Tabular Deep Learning

1 Introduction

Machine learning (ML) has achieved significant success in effectively detecting malware and securing computer systems. However, in real-world settings, the malware ecosystem is highly dynamic; large volumes of new malware frequently emerge. Given that AV-TEST reports roughly 500K new malware and potentially unwanted applications (PUAs) each day, and VirusTotal processes about two million new submissions daily [22], regularly retraining models becomes essential. However, retraining on accumulated malware samples incurs extremely high

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
R. C. Wong et al. (Eds.): PAKDD 2026, LNAI 16599, pp. 547–559, 2026.
https://doi.org/10.1007/978-981-92-1465-5_43

computational and storage requirements [16], highlighting the need for a practical solution that can continually detect new malware variants while reducing the volume of the training set.

To address this, defenders may resort to discarding older samples or ignoring newer ones, neither of which is practical. A more effective approach is continual learning (CL), which enables models to learn from a continuous data stream (treated as sequential tasks) without requiring full retraining [14,15]. To avoid constructing such a joint training dataset, CL strategies employ several mechanisms: replaying previous samples using generators or a limited set of stored exemplars [17,21], adapting important weights from prior tasks [8], or allocating isolated parameters for each task [12,19].

Research efforts attempting to directly apply vision-oriented CL methods to malware detection have shown poor performance [16] due to the uniquely structured feature semantics, high intra-class variance, and non-stationary distributions of malware data. These observations highlight the importance of understanding the unique structure of the malware feature space. To address this challenge, MalCL [14] employs Generative Replay (GR) with Generative Adversarial Networks (GANs) to synthesize samples from previous tasks and retrain the model on both generated and new samples. By incorporating feature matching and replay sample selection to improve generation quality, MalCL achieves state-of-the-art performance.

Despite its contributions, MalCL does not address several key challenges in malware classification. First, malware features are high-dimensional and tabular, comprising concatenated fields with highly variable scales, which necessitates CL methods specifically tailored to such unique tabular feature structures. Furthermore, these features include high-dimensional hashed string fields with extremely variable value ranges, which further complicates sample generation [1]. Nevertheless, MalCL does not examine the structural properties of malware features and instead adopts a naive GAN architecture, making only minimal architectural adjustments without a deeper understanding of the underlying feature space. Moreover, malware datasets are often highly imbalanced across families due to the difficulty of collecting rare samples [10]. Although imbalance-aware approaches are necessary, prior work does not address this aspect.

To address these limitations, we propose TabCL (Tabular-Aware Continual Learning), a CL framework tailored for malware family classification under class-incremental learning (Class-IL) settings, where new malware families continually emerge. We also consider a time-incremental learning (Time-IL) scenario, in which temporal gaps exist even within the same family. To the best of our knowledge, TabCL is the first to integrate a tabular generative model, Conditional Tabular GAN (CTGAN) [23], into continual malware analysis, enabling more realistic replay by separately modeling continuous and categorical features. To improve efficiency and stability in TabCL, we introduce SiMBER, a SimHash-based [13] compression method that maps high-cardinality string fields to compact, locality-sensitive binary vectors. This transformation preserves semantic similarity while reducing the wide and sparse value ranges of tabular

malware features. SiMBER reduces training costs without sacrificing accuracy and improves performance across GR-based models such as MalCL and TabCL.

Our contributions in this study are: (1) **SiMBER**, a feature compression technique for malware features that reduces dimensionality by approximately 60% and training time by 2.5×, while preserving or improving performance by up to 19%, and (2) **TabCL**, A CL framework for malware family classification that integrates tabular-aware generative model-based replay and SiMBER feature compression. Extensive evaluation on the EMBER 2018 [1], EMBER 2024 [6], and MalDICT [7] datasets shows that TabCL outperforms MalCL [14] by up to 9–16% in the Class-IL multi-class and binary classification settings and achieves comparable performance in the Time-IL setting.

2 Related Work

CL Approaches. The key challenge in CL is catastrophic forgetting (CF)– a phenomenon in which a ML model rapidly loses previously learned knowledge when trained on new data [4]. A substantial body of prior work has explored various strategies to mitigate CF, including regularization [8], parameter isolation [12,19], and interpolation [9]. Among these, replay-based methods have emerged as practical and widely adopted solutions in CL settings [11,15,17,18,21]. In general, replay-based approaches mitigate forgetting by retraining the classifier on samples that approximate the distributions of previously seen data. These methods can be categorized into three types: Experience Replay (ER) [18], GR [21], and distillation-based replay. ER techniques, such as iCaRL [17], explicitly store and replay a subset of historical data retained in a limited memory buffer $\mathcal{M}$. In contrast, GR employs generative neural networks to synthesize samples that emulate the distribution of prior tasks. Finally, distillation-based methods, such as LwF [11], align the current model's predictions with the soft targets generated by earlier models.

CL in Malware Domain. Although CL has been comprehensively studied across various domains, its application to the malware domain remains limited. The first attempt by Rahman et al. [16] showed that Computer Vision (CV)-based CL methods are largely ineffective at preventing CF in the malware domain. To address these limitations, subsequent studies have proposed various approaches to integrate CL into the malware domain. MalCL [14] adopted a GAN-based malware detector for the Class-IL setting and enhanced the GR approach by combining Feature Matching Loss (FML) and replay sample selection based on the intermediate layer of the classifier. This approach enabled effective generation of prior task data, outperforming CV-based CL models. MalFSCIL proposed a variational autoencoder (VAE)-based GR method tailored for few-shot Class-IL, enabling stable class expansion even in few-shot scenarios [2]. Furthermore, Rahman et al. [15] propose MADAR, a distribution-aware replay mechanism that accounts for both inter-family and intra-family variations in malware datasets. MADAR improves accuracy and scalability by balancing representative samples and outliers in the raw feature space.

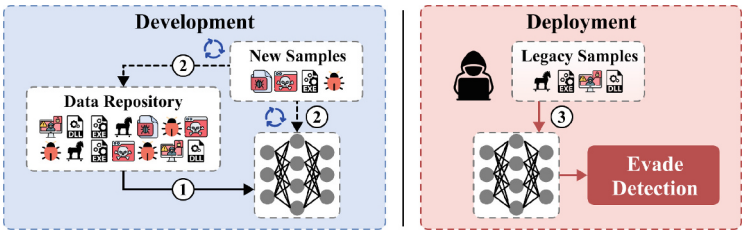


Fig. 1. Threat model TabCL: Retrograde Malware Attack (RMA)—the attacker reuses the legacy malware sample to evade the detector.

Nevertheless, these approaches *do not handle the tabular structures and task imbalance of the malware features*. As such, we propose a tabular-aware CL framework to mitigate CF.

3 Threat Model

Figure 1 illustrates the threat model considered in this work. In a typical deployment setting, an ML-based malware detection system is initially trained on a repository of labeled malware and benign samples (Fig. 1 ①) and deployed for real-time classification. As new threats emerge, the system is periodically updated with recent malware samples to maintain performance in the face of an evolving threat landscape (Fig. 1 ②). This continual update process is essential but introduces a critical vulnerability – without mechanisms to retain past knowledge, the model tends to forget the representations of earlier threats due to CF. Consequently, the system becomes biased toward recently seen samples, a problem exacerbated by task recency effects. This vulnerability gives rise to the *Retrograde Malware Attack (RMA)*. As shown in Fig. 1 ③, an adversary can bypass detection by reintroducing older or slightly modified malware variants that the system no longer recognizes. The likelihood of such evasion increases with each update cycle in the absence of replay mechanisms. RMA underscores a central challenge in CL-based malware detection, maintaining consistent performance on previously known threats while adapting to new ones.

4 Heterogeneous Tabular Generative CL

EMBER features [1] are among the most widely used Windows malware feature sets for benchmarking in both academia and industry [14,15,20]. Accordingly, we focus our analysis on the tabular features represented in this feature space, enabling consistent evaluation and direct comparison with prior studies. Given the broad adoption of EMBER within the security community, we believe that studying this feature set provides strong justification and motivation for our proposal. The EMBER dataset contains a heterogeneous mix of features extracted

from PE files, including both continuous and categorical variables. Such diversity poses challenges for generative models that are not explicitly designed to handle mixed-type tabular data. Key feature types in EMBER and their transformations in TabCL are:

- **Numerical Features**: Continuous values derived from statistical or structural properties of binaries, such as entropy, number of execution paths, or section sizes. For these values, we apply a mode-specific normalization based on a Variational Gaussian Mixture Model (VGM) in CTGAN [23] to transform each continuous feature into a distributionally uniform space. This allows the generator to capture complex, multi-modal numerical distributions effectively.
- **Categorical Features**: Discrete-valued features, often with either low or high cardinality, include indicators such as the presence of debug information, `DLL` characteristics, and subsystem types. We adopt CTGAN's encoding strategy, representing categorical values with one-hot encodings. This approach allows the model to capture category-specific distributions more effectively.
- **Hashed Strings**: String-based fields (e.g., section names or characteristics) are typically represented using hash-encoded categorical formats. However, we observe that the columns generated from these string-derived hash features are not mutually independent, and their dimensionality is extremely high, making them unsuitable for standard one-hot encoding. To address this issue, we substitute SimHash [13] for Scikit-Learn's original `FeatureHasher`, leveraging its binary locality-sensitive hashing properties to produce a more compact and semantically meaningful representation (see Sect. 5 for details)

Instead of using the original EMBER dataset representation, which simply concatenates all features into a single 1D vector, we focus on reliably modeling the joint distribution of heterogeneous features. The previous approach, while convenient, neglects the fundamentally different statistical properties of continuous and categorical attributes. Nevertheless, many prior studies [3,5,14] continue to adopt it when applying ML or generative models to malware data. In contrast, we employ a tabular-aware generative model that explicitly handles both discrete and continuous features. To keep the dimensionality of the conditional vector manageable, we compress the features by reducing both the number of categorical columns and their cardinalities. This compression preserves core semantic information while lowering dimensionality, enabling more tractable and more generalizable generative modeling of malware features.

5 TabCL Framework

CL Pipeline Overview. As illustrated in Fig. 2, TabCL operates in a task-incremental learning (Task-IL) setting with N tasks. We consider two scenarios: Class-IL, where new malware families appear sequentially, and Time-IL, where

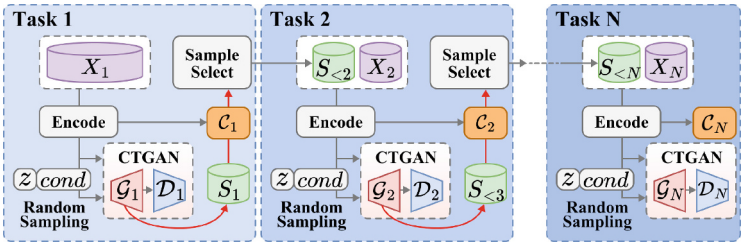


Fig. 2. TabCL pipeline.

new samples from existing families emerge over time. In the multi-class Class-IL setting, the first task ($t = 1$) contains a real sample set X_1 from 50 families (30 families in the case of MalDICT dataset), and each subsequent task ($t \in 2, \ldots, 11$) introduces new samples from five previously unseen families. In the binary Class-IL setting, the first task ($t = 1$) contains a real sample set X_1 from 50 malware families and real benign samples, and each subsequent task ($t \in 2, \ldots, 11$) introduces new samples from five previously unseen malware families and the benign samples. The number of malware and benign samples is the same for each task. In the Time-IL setting, the first task includes samples from 100 families, and subsequent tasks ($t \in 2, \ldots, 6$) introduce new samples from the same families. The first task contains 7 months, and the time gap between each task is one month, totaling 6 tasks.

For each task t, a classifier $\mathcal{C}_t$ is trained jointly with a generator $\mathcal{G}_t$ and discriminator $\mathcal{D}_t$. Prior to training, real samples are encoded into a format suitable for conditional generation. A class condition vector $\mathbf{cond}_t$ is sampled to indicate the target class and concatenated with a noise vector $\mathbf{z}_t$ to form the input to $\mathcal{G}_t$. The generator produces synthetic samples $S_t = \mathcal{G}_t(\mathbf{z}_t, \mathbf{cond}_t)$, while the discriminator $\mathcal{D}_t$ learns to distinguish S_t from real samples X_t. After convergence, $\mathcal{C}_t$ is updated using X_t. Starting from $t \geq 2$, it also incorporates replay samples $S_{<t}$ generated by earlier $\mathcal{G}_{<t}$ to retain knowledge of previous tasks. A representative subset of generated samples is selected and stored after each task for replay. Each new task is trained on the union of X_t and the stored replay samples. The overall adversarial training procedure used in TabCL is provided in Algorithm 1.

Feature Representation. To enable the modeling of the joint distribution of mixed-type tabular data in the generator $\mathcal{G}$, TabCL employs CTGAN, a tabular-aware generative model, as its generator. To adapt CTGAN to malware features, we (1) analyzed the characteristics of each column and separately encoded *continuous* and *discrete* columns using specialized transformations, and (2) separated hashed strings into a new column type, *discrete-sequence*, to improve training efficiency in terms of both time and accuracy. To be specific, we used different techniques for heterogeneous features:

– Continuous column $\boldsymbol{C_i}$: For the i-th continuous column C_i, we applied mode-specific normalization in CTGAN. We first fit the column using a Gaussian

Algorithm 1. Training a GAN in TabCL

Input: Training dataset $X = \{x_i, y_i\}_{i=1}^{N}$
Initialize $\mathcal{G}$ and $\mathcal{D}$ networks with random weights
Define the number of epochs E, batch size m, and the number of iterations k
for $epoch = 1$ to E **do**
  for each batch B in X **do**
    for $iteration = 1$ to k **do**
      Sample noise $z_i \sim \mathcal{N}(0, 1)$ where $i = 1, \ldots, m$
      Sample condition vector $cond \sim \{D_1, ..., D_{N_d}\}$
      Sample real instances $x_i \sim p_X(x \mid cond)$ where $i = 1, \ldots, m$
      Generate synthetic samples x_i' where $x_i' = \mathcal{G}(z_i \oplus cond)$ and $i = 1, \ldots, m$
      Update $\mathcal{D}$ by ascending its stochastic gradient with gradient penalty $\mathcal{L}_{\mathrm{GP}}$:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} [\mathcal{D}(x_i) - \mathcal{D}(x_i') + \lambda_{GP} \cdot \mathcal{L}_{\mathrm{GP}}]$$

    end for
    Sample noise $z_i \sim \mathcal{N}(0, 1)$ where $i = 1, \ldots, m$
    Sample condition vector $cond \sim \{D_1, ..., D_{N_d}\}$
    Generate synthetic samples x_i' where $x_i' = \mathcal{G}(z_i \oplus cond)$ and $i = 1, \ldots, m$
    Update $\mathcal{G}$ by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} [-\log(\mathcal{D}(x_i')) + \lambda_{CE} \cdot \mathrm{CE}(cond, x_i')]$$

  end for
end for

Mixture Model (GMM) with $\boldsymbol{m_i}$ components through a VGM approximation. Given a value $\boldsymbol{c_{i,j}}$, representing the i-th continuous column of the j-th row, CTGAN computes the probability of each mode as $\boldsymbol{\rho_k = \mu_k \mathcal{N}(c_{i,j}; \eta_k, \phi_k)}$, where μ_k is the mixture weight, and $\boldsymbol{\eta_k}$ and $\boldsymbol{\phi_k}$ denote the Gaussian mean and Gaussian standard deviation, respectively. A mode index k is then sampled, and the value is encoded using a one-hot mode indicator $\boldsymbol{\beta_{i,j} \in \{0, 1\}^{m_i}}$ and a normalized scalar $\alpha_{i,j} = \frac{c_{i,j} - \eta_k}{4\phi_k}$ within the selected mode. This approach enables effective modeling of multimodal and non-Gaussian distributions. We constrain the number of clusters in the VGM model to match the number of malware families to make the learned representation captures inter-class variability while avoiding overfitting to fine-grained fluctuations.

- **Discrete column $\boldsymbol{D_i}$**: We represented i-th categorical column $\boldsymbol{D_i}$ using a one-hot vector $\mathbf{d_{i,j}}$, corresponding to the i-th discrete column of the j-th row, Additionally, we incorporate CTGAN's conditional generation mechanism, which samples a condition $\boldsymbol{D_{i^*} = k^*}$ and learns the conditional distribution $\boldsymbol{\mathbb{P}_{\mathcal{G}}(row \mid D_{i^*} = k^*)}$ to ensure that the generated samples satisfy the condition. This Training-by-Sampling strategy selects conditions proportional to the log-frequency of categories, ensuring sufficient representation of minority classes during training and mitigating class imbalance. This column includes low-cardinality features such as machine IDs and subsystem IDs, which can be a compact and interpretable representation with one-hot encoding.
- **Discrete-Sequence column $\boldsymbol{H_i}$**: Critically, we observe that the columns derived from string-derived hash features are not mutually independent and

exhibit extremely high dimensionality, making standard one-hot encoding infeasible. To address this, we replace the original `FeatureHasher` (of Scikit-Learn library) with SimHash [13], a binary locality-sensitive hashing technique. Given a set of strings $S_1, S_2, \ldots, S_m$, SimHash first applies a base hash function (e.g., MD5) to each string to produce a binary vector $\mathbf{s}_j \in \mathbf{\{0,1\}}^n$ for each $\boldsymbol{S}_j$. Each bit is then mapped to $\mathbf{\{-1,+1\}}$, yielding a transformed vector $\tilde{\mathbf{s}}_j \in \mathbf{\{-1,+1\}}^n$ defined by $\tilde{\mathbf{s}}_j[i] = -1$ if $\mathbf{s}_j[\boldsymbol{i}] = \mathbf{0}$ and $\tilde{\mathbf{s}}_j[i] = +1$ otherwise. For each bit position $i \in \{1, \ldots, n\}$, we compute an aggregate score $v_i = \sum_{j=1}^{m} \tilde{\mathbf{s}}_j[i]$, which captures the dominant bit orientation across all hashed strings, implicitly encoding semantic similarity. Finally, the n-bit SimHash vector $\mathbf{h} \in \mathbf{\{0,1\}}^n$ is obtained via thresholding, where $\mathbf{h}[\boldsymbol{i}] = \mathbf{1}$ if $v_i > 0$ and $\mathbf{h}[i] = 0$ otherwise.

Each data row $\mathbf{r}_j$ is constructed by concatenating type-aware representations of all columns – i.e., normalized scalars and mode indicators for continuous features, one-hot vectors for discrete ones, and a hash vector for the discrete-sequence features, yielding the following representation:

$$\mathbf{r}_j = \alpha_{1,j} \oplus \beta_{1,j} \oplus \cdots \oplus \alpha_{N_c,j} \oplus \beta_{N_c,j} \oplus \mathbf{d}_{1,j} \oplus \cdots \oplus \mathbf{d}_{N_d,j} \oplus \mathbf{h}_{1,j} \oplus \cdots \oplus \mathbf{h}_{N_s,j} \quad (1)$$

Here, N_c, N_d and N_s denote the number of continuous, discrete, and discrete-sequence columns, respectively. The resulting vector $\mathbf{r}_j$ serves as input to both the generator and the discriminator in adversarial training. Additionally, we adapted CTGAN's Training-by-Sampling strategy where the model randomly selects a discrete column and samples a value from it to condition on, producing a *cond* vector for the input of the $\mathcal{G}$ (See Algorithm 1). This conditional strategy ensures balanced exposure to all categories and reduces the risk of mode collapse.

Our simplified variant of the EMBER feature representation, referred to as SiMBER, is designed to reduce dimensionality while preserving critical information for malware classification. This simplification is instrumental in improving the training efficiency of generative models such as CTGAN.

Replay Sample Selection. As crucial as the quality of the generative model itself is the ability to select high-quality samples from its generated outputs. To achieve this, we adopted per-class mean logit-based matching following the CMean sampling from MalCL [14], replacing the conditional vector sampling based on CTGAN's conditioning mechanisms. Let $\mathcal{L}_i(x)$ denote the logit vector extracted from the middle layer of C_i for input x at task i. For each class c, we compute the class-wise mean logit vector as follows:

$$\bar{\mathcal{L}}_{i,c}(x) = \frac{1}{n_c} \sum_{x \in X_c} \mathcal{L}_i(x) \quad (2)$$

where $X_c \subseteq X_i$ is the set of real samples with label c from task i, and $n_c = |X_c|$. Then, among a set of synthetic samples S, we select the top-k samples whose logits are closest to the class mean:

$$S_{c,k} = \left\{ s \mid \arg\min_{s} \left\| \mathcal{L}_i(s) - \bar{\mathcal{L}}_{i,c}(x_j) \right\|, \forall s \in S, \forall x \in X_c \right\} \tag{3}$$

This logit-based filtering ensures that the selected replay samples are semantically aligned with the original class distributions, thereby improving representativeness and stability in CL training.

6 Experimental Results

Malware Datasets. We used three renowned large-scale Windows PE-based benchmarks, EMBER 2018 [1], EMBER 2024 [6], and MalDICT [7]. EMBER 2018 provides rich continuous and discrete features, including entropy, libraries, and imported/exported functions. EMBER 2024 contains over 3.2 million recent malware samples with extended features, including DOS/Rich header and string pattern statistics. MalDICT contains over 4 million VirusShare samples in EMBER 2018 format. We used the 100 malware families with the most samples.

Evaluation Metrics. To summarize performance across tasks from task 0 through task t, we report two metrics: (1) *Mean*, the average accuracy over all tasks, and (2) *Minimum (Min)*, the lowest accuracy across all tasks. Aligned with standard CL protocols [14,16], these metrics evaluate the model's ability to retain knowledge of earlier tasks while learning new ones, providing insight into both overall performance and worst-case forgetting.

Performance Bounds. Following prior work, we compare TabCL against two baseline settings [14,16], *None* and *Joint*. The *None* setting trains only on data from the current task without any replay or synthetic augmentation, thereby serving as a lower-bound. In contrast, the *Joint* setting assumes access to all data from previous tasks, enabling upper-bound performance through full retraining.

Settings. All experiments are conducted on a server equipped with 8× NVIDIA RTX A6000 GPUs and 1.0 TiB RAM, running Ubuntu 20.04.5 LTS with CUDA 11.4. For fair comparison, we implemented all baseline models, adopted their original hyperparameter settings, and evaluated each method using five independent seeds, randomizing the order of classes or samples in subsequent tasks. For runtime comparison, we conducted evaluations on a dedicated server without background processes to ensure fairness. Full hyperparameter specifications are detailed in the source code.

6.1 Class-IL and Time-IL Experiments

Table 1 reports the mean and worst-case (minimum) accuracy in the Class-IL scenario using the EMBER 2018 dataset. Notably, TabCL outperformed all baseline models, achieving a substantial improvement of more than 10% over the state-of-the-art model in both metrics.

Table 1. Performance comparison of TabCL against baselines on the EMBER 2018 dataset in Class-IL (multi-class) scenarios.

Model	Ours	Baseline							
	TabCL	**None**	**Joint**	**EWC**	**GR**	**MalCL**	**LwF**	**TAMiL**	**iCaRL**
Mean	**66**	34	87	26	27	55	11	33	55
Min	**43**	9	83	8	10	33	1	15	30

Table 2. Comparison of TabCL and baselines on EMBER 2018, EMBER 2024, and MalDICT in various settings.

Setting / Model	Class-IL (Multi)						Class-IL (Binary)		Time-IL (Multi)	
	EMBER'18		EMBER'24		MalDICT		EMBER'18		EMBER'18	
	Mean	Min	Mean	Min	Mean	Min	Mean	Min	Mean	Min
None	34	9	33	14	30	11	89	85	73	63
Joint	87	83	80	75	72	65	97	97	78	72
MalCL	55	33	46	24	42	22	88	82	**78**	**73**
TabCL	**66**	**43**	**55**	**35**	**49**	**32**	**99**	**98**	76	**73**

Furthermore, we selected the most competitive baseline, MalCL, to compare classification accuracy across Class-IL scenarios for both multi-class and binary classification, as detailed in Table 2. In the multi-class setting, TabCL consistently outperforms MalCL by 7–11 % on EMBER 2018, EMBER 2024, and MalDICT in both metrics, demonstrating substantially better knowledge retention across different datasets. In the binary classification setting, TabCL also shows 11% and 16% improvements over MalCL in mean and minimum accuracy on EMBER 2018, respectively. Notably, the worst-case accuracy is only one percentage point lower than the mean, indicating highly stable performance across tasks. We attribute these improvements primarily to CTGAN's tabular-aware modeling, which enables more reliable replay by separately handling continuous and discrete features.

To evaluate TabCL and the baselines under the natural concept drift of malware, we further conducted Time-IL experiments in which temporal drift occurs even within the same family. As shown in Table 1, the performance gap between None and Joint becomes smaller than in the Class-IL setting. Despite this, TabCL remains effective, achieving performance comparable to MalCL and exceeding None by 10% points on EMBER 2018.

6.2 SiMBER Performance Evaluation

We evaluate the performance improvements and training complexity introduced by SiMBER when applied to the baselines, MalCL, and TabCL. Table 3 reports the mean accuracy over all tasks for both the original EMBER features and

Table 3. Mean classification accuracy and training time comparison using EMBER and SiMBER feature formats on the EMBER 2018 dataset.

Dataset	Model	Evaluation	Feature Format	
			EMBER	SiMBER
EMBER 2018 [1]	None	Mean Accuracy	28	**30**
	Joint		**89**	87
	MalCL		55	**56**
	TabCL		47	**66**
	TabCL	Training Time (s)	24,480	**9,960 (59% ↓)**

Table 4. Comparison between EMBER and SiMBER in terms of dimensionality and collision rate in the EMBER 2018 and EMBER 2024 datasets.

Dataset	Evaluation	Feature Format	
		EMBER	SiMBER
EMBER 2018 [1]	Mean Collision Rate	15.20%	**7.06% (8.14%p ↓)**
EMBER 2018 [1]	Dimensionality	2,381	**1,006 (57% ↓)**
EMBER 2024 [6]		2,568	**1,251 (51% ↓)**

SiMBER on the EMBER 2018 and EMBER 2024 datasets. Although SiMBER reduces the input dimensionality by more than 50% (See Table 4), it achieves accuracy comparable to (and in some cases up to 19% higher than) that of the original EMBER features across all evaluated models.

As shown in Table 3 and 4, SiMBER reduces the feature dimensionality of EMBER 2018 and EMBER 2024 by 57% and 51%, respectively, resulting in a 59% reduction in training time and a 2.5× speedup compared to training on the original EMBER features. Notably, SiMBER reduced the mean collision rate averaged across all discrete-sequence columns by 8.14% points. Overall, SiMBER preserves classification performance while substantially reducing computational costs.

7 Conclusion and Future Work

We presented TabCL, a CL framework for malware classification that enables structure-aware CL with a tabular-aware generative model and a lightweight feature representation (SiMBER). TabCL improves accuracy over MalCL by up to 11%, while reducing training costs and improving training stability. For future work, we aim to extend TabCL to diverse real-world malware datasets and feature representations beyond EMBER.

Acknowledgements. This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. RS-2025- 02263915, RS-2025–16071657, RS-2026–25498869).

References

1. Anderson, H.S., Roth, P.: EMBER: an open dataset for training static PE malware machine learning models (2018). arXiv:1804.04637 arXiv preprint
2. Chai, Y., et al.: Malfscil: a few-shot class-incremental learning approach for malware detection. IEEE Trans. Inf. Forensics Secur (TIFS) (2025)
3. Eren, M.E., et al.: Catch'em all: classification of rare, prominent, and novel malware families. In: International Symposium on Digital Forensics and Security (ISDFS) (2024)
4. French, R.M.: Catastrophic forgetting in connectionist networks. Trends Cogn. Sci., (1999)
5. Harang, R., Rudd, E.M.: SOREL-20M: a large scale benchmark dataset for malicious PE detection, (2020). arXiv:2012.07634 arXiv preprint
6. Joyce, R.J., et al.: EMBER2024-a benchmark dataset for holistic evaluation of malware classifiers. In: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD) (2025)
7. Joyce, R.J., Raff, E., Nicholas, C., Holt, J.: MalDICT: benchmark datasets on malware behaviors, platforms, exploitation, and packers (2023). arXiv:2310.11706 arXiv preprint
8. Kirkpatrick, J., et al.: Overcoming catastrophic forgetting in neural networks. Proc. Natl. Acad. Sci.(2017) PNAS
9. Kozal, J., Wasilewski, J., Krawczyk, B., Woźniak, M.: Continual learning with weight interpolation. In: Conference on computer vision and pattern recognition (CVPR), pp. 4187–4195 (2024)
10. Li, T., et al.: A malware detection model based on imbalanced heterogeneous graph embeddings. Expert Syst. Appl. (2024)
11. Li, Z., Hoiem, D.: Learning without forgetting. IEEE Trans. Pattern Anal. Mach. Intelli. (TPAMI) **40**(12), 2935–2947 (2017)
12. Mallya, A., Lazebnik, S.: Packnet: Adding multiple tasks to a single network by iterative pruning. In: IEEE conference on Computer Vision and Pattern Recognition (CVPR) (2018)
13. Manku, G.S., Jain, A., Das Sarma, A.: Detecting near-duplicates for web crawling. In: International Conference on World Wide Web (2007)
14. Park, J., Ji, A., Park, M., Rahman, M.S., Oh, S.E.: MalCL: leveraging GAN-based generative replay to combat catastrophic forgetting in malware classification. In: AAAI Conference on Artificial Intelligence (2025)
15. Rahman, M.S., Coull, S., Yu, Q., Wright, M.: MADAR: efficient continual learning for malware analysis with distribution-aware replay. In: Conference on Applied Machine Learning in Information Security, CAMLIS (2025)
16. Rahman, M.S., Coull, S.E., Wright, M.: On the limitations of continual learning for malware classification. In: First Conference on Lifelong Learning Agents (CoLLAs) (2022)
17. Rebuffi, S.A., Kolesnikov, A., Sperl, G., Lampert, C.H.: IcArl: Incremental classifier and representation learning. In: IEEE conference on Computer Vision and Pattern Recognition (CVPR) (2017).

18. Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T., Wayne, G.: Experience replay for continual learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2019)
19. Rusu, A.A., et al.: Progressive neural networks (2016). arXiv:1606.04671 arXiv preprint
20. Severi, G., Meyer, J., Coull, S., Oprea, A.: {Explanation-Guided} backdoor poisoning attacks against malware classifiers. In: USENIX Security Symposium (2021)
21. Shin, H., Lee, J.K., Kim, J., Kim, J.: Continual learning with deep generative replay. Adv. Neural Inf. Process. Syst. (NeurIPS) (2017)
22. VirusTotal: virustotal – stats (2023). https://www.virustotal.com/gui/stats
23. Xu, L., Skoularidou, M., Cuesta-Infante, A., Veeramachaneni, K.: Modeling tabular data using conditional GAN. In: Advances in Neural Information Processing Systems (NeurIPS) (2019)